FACULTAD DE INGENIERÍA

Escuela Académico Profesional de Ingeniería de Sistemas e Informática

Proyecto de Investigación

Evaluación del efecto de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa

AUTORES:

Harold Huanca Ccasa

Jose Leonardo Cruz Gonzales

Wilder Ronald Ccasani Quispe

Cusco, Perú

2026

ÍNDICE

INTRODUCCIÓN III

1. EL PROBLEMA DE INVESTIGACIÓN 1

1.1. Planteamiento del Problema . . . 1
1.2. Formulación del Problema . . . 3
1.2.1. Problema General . . . 3
1.2.2. Problemas Específicos . . . 3
1.3. Objetivos de la Investigación . . . 3
1.3.1. Objetivo General . . . 3
1.3.2. Objetivos Específicos . . . 3
1.4. Justificación de la Investigación . . . 4
1.4.1. Justificación Teórica . . . 4
1.4.2. Justificación Práctica . . . 4
1.4.3. Justificación Metodológica . . . 4
1.5. Delimitación de la Investigación . . . 5
1.5.1. Delimitación Espacial . . . 5
1.5.2. Delimitación Temporal . . . 5
1.5.3. Delimitación Social . . . 5
1.6. Hipótesis y Variables . . . 5
1.6.1. Hipótesis . . . 5
1.6.2. Definición de Variables . . . 6
1.6.3. Operacionalización de Variables . . . 6

2. MARCO TEÓRICO 7

2.1. Antecedentes de la investigación . . . 7
2.1.1. Antecedentes Internacionales . . . 7
2.1.2. Antecedentes Nacionales . . . 9
2.2. Bases teóricas . . . 10
2.2.1. Marco Conceptual . . . 10
2.2.1.1. Ingeniería de Prompts (Prompt Engineering) . . . 10
2.2.1.2. Mantenibilidad del Software . . . 11
2.2.1.3. Deuda Técnica (Technical Debt) . . . 11
2.2.1.4. Complejidad Ciclomática de McCabe . . . 11
2.2.1.5. PromptDebt . . . 11
2.2.1.6. Análisis Estático de Código . . . 11
2.2.1.7. Deuda Técnica Autoadmitida (SATD) . . . 11
2.2.2. Bases Teóricas . . . 12
2.2.2.1. Ingeniería de Software y Calidad del Producto . . . 12
2.2.2.2. Inteligencia Artificial Generativa en el Desarrollo de Software 12
2.2.2.3. Paradigma de la Ingeniería de Prompts (Prompt Engineering) 13
2.2.2.4. Deuda Técnica en Entornos Generativos . . . 13

2.2.2.5. Marcos de Trabajo y Evaluación Automatizada 14

3. METODOLOGÍA **15**
3.1. Método, tipo y alcance de la investigación 15
3.1.1. Método de investigación 15
3.1.2. Tipo de investigación 15
3.1.3. Alcance de la investigación 15
3.1.4. Diseño de la investigación 15
3.1.5. Enfoque de la investigación 16
3.2. Materiales y métodos 16
3.2.1. Materiales 16
3.2.2. Recursos de Infraestructura y Hardware 16
3.2.2.1. Estación de Trabajo de Alto Rendimiento 16
3.2.2.2. Entorno de Ejecución y Nodos Computacionales 16
3.2.3. Recursos Lógicos y Modelos de Inteligencia Artificial 17
3.2.3.1. Modelos de Lenguaje de Gran Escala (LLMs) de Acceso Abierto 17
3.2.3.2. Entorno de Orquestación y Contenerización 17
3.2.4. Herramientas de Análisis Estático y Desarrollo 18
3.2.4.1. Framework de Evaluación de Calidad de Código 18
3.2.4.2. Ecosistema de Lenguajes de Programación Objetivo 18
3.2.5. Planificación del Proyecto 18
3.2.5.1. Presupuesto 18
3.2.5.2. Cronograma de Actividades 19
3.2.6. Métodos 21
3.2.7. Ingeniería de Prompts y Diseño del Framework 21
3.2.7.1. Conceptualización del Framework basado en ISO/IEC 25010 21
3.2.7.2. Aplicación de Patrones Arquitectónicos de Prompts 21
3.2.8. Desarrollo y Generación de Código de Microservicios 21
3.2.8.1. Configuración del Entorno de Generación 21
3.2.8.2. Ejecución de Pruebas de Generación 21
3.2.9. Pipeline de Análisis Estático y Validación de Calidad 22
3.2.9.1. Proceso de Evaluación de Métricas de Código 22
3.2.9.2. Validación y Control de Calidad 22
3.3. Sustento Teórico de la Metodología 22

4. RESULTADOS Y DISCUSIÓN **23**
4.1. Presentación de resultado 23
4.2. Discusión de resultados 23

5. CONCLUSIONES Y RECOMENDACIONES **24**

ANEXOS **25**

Referencias Bibliográficas **26**

INTRODUCCIÓN

CAPÍTULO 1
EL PROBLEMA DE INVESTIGACIÓN

1.1 Planteamiento del Problema

A nivel global, la integración de la Inteligencia Artificial Generativa (IAG) en el ciclo de vida del desarrollo de software ha transformado la productividad de la industria; no obstante, ha introducido un fenómeno crítico denominado "deuda técnica automatizada". Investigaciones recientes como la de Hou et al. [1] sostienen que el código generado por modelos de lenguaje de gran escala (LLMs) a menudo carece de una visión arquitectónica integral, derivando en sistemas con una complejidad estructural elevada. Esta tendencia sugiere un incremento sustancial en la velocidad de codificación, pero a costa de una degradación en la mantenibilidad, lo que contraviene los estándares de calidad interna establecidos por la norma ISO/IEC 25010 [2]. Diversos estudios han evidenciado que el código generado por modelos de lenguaje de gran escala presenta limitaciones en términos de mantenibilidad y calidad estructural. Por ejemplo, investigaciones recientes señalan que el uso de LLMs en generación de código puede introducir complejidad innecesaria y deuda técnica acumulada, especialmente cuando no se aplican lineamientos estructurados de diseño o validación [3, 4]. Estos hallazgos evidencian la necesidad de establecer marcos metodológicos que orienten la generación de código hacia estándares de calidad definidos.

En el contexto nacional, el Perú atraviesa un proceso de aceleración digital orientado a la optimización de procesos mediante la automatización. Según el [5], existe un incremento sustancial en la adopción de servicios tecnológicos en el sector empresarial; sin embargo, este avance presenta desafíos estructurales. De acuerdo con la [6], el país enfrenta una brecha en la formación de talento especializado y en la definición de estándares de gobernanza técnica que aseguren la calidad de las soluciones de IA. Esta carencia de marcos de validación estandarizados eleva el riesgo de acumular deuda técnica en sistemas críticos, comprometiendo la mantenibilidad y la seguridad del software desarrollado en el entorno nacional.

A nivel local, se observa en el entorno académico y profesional de la región Cusco una adopción empírica de la Inteligencia Artificial Generativa para la resolución de problemas de programación. A través de la experiencia en el desarrollo de proyectos de software dentro de la Universidad Continental, se ha identificado que la prioridad suele centrarse en la funcionalidad inmediata, postergando la calidad interna del código. Esta observación coincide con las brechas identificadas en el [7], donde se evidencia que la transformación digital de la región requiere de una mayor rigurosidad en los estándares técnicos. Por lo tanto, la presente investigación surge de la necesidad de establecer herramientas que transformen esta práctica empírica observada en la formación profesional en un proceso estandarizado que asegure la mantenibilidad según la norma ISO/IEC 25010.

El problema central se identifica como la insuficiente calidad estructural y la elevada

deuda técnica presente en el software generado mediante Inteligencia Artificial Generativa (IAG) ante la ausencia de marcos metodológicos de ingeniería. Investigaciones previas [3] demuestran que, aunque los modelos de lenguaje producen código funcional, este suele presentar deficiencias en los atributos de mantenibilidad y complejidad ciclomática definidos por la norma ISO/IEC 25010. Asimismo, la carencia de un método sistemático para estructurar los requerimientos técnicos o prompts deriva en lo que la literatura denomina como opacidad técnica o "cajas negras" de código [8], dificultando los procesos de auditoría, optimización y escalabilidad del software en entornos de producción.

Las causas de esta problemática son multidimensionales. En el plano técnico, se observa una dependencia de modelos de IA que, a pesar de su capacidad funcional, generan un incremento del 34 % en la complejidad ciclomática y una redundancia lógica 2.1 veces superior a la del código humano [9], evidenciando que los LLMs requieren de una guía estructural y arquitectónica para asegurar la mantenibilidad [10]. A nivel organizacional, la presión por reducir los tiempos de entrega (*Time-to-Market*) fomenta la acumulación de deuda técnica autoadmitida (*Self-Admitted Technical Debt* - SATD) y la omisión de fases críticas de validación [11, 12]. Finalmente, en el factor humano, persiste una carencia de competencias en ingeniería de prompts aplicada sistemáticamente, lo que deriva en una degradación de la calidad denominada *PromptDebt* [4], limitando el uso de la IA a una herramienta de consulta en lugar de integrarla como un componente robusto en un *pipeline* de producción profesional [13, 14].

Las consecuencias de persistir en este modelo empírico de desarrollo son graves. Operativamente, las organizaciones enfrentarán un incremento exponencial en los costos de mantenimiento preventivo y correctivo, debido a que el código generado por IA sin supervisión técnica demanda un esfuerzo de refinamiento masivo para alcanzar estándares profesionales [15]. Económicamente, la deuda técnica acumulada puede superar el valor del activo de software, pues el incremento en la complejidad y la redundancia lógica elevan los costos de evolución del sistema hasta volverlo insostenible [9, 16]. Socialmente, la baja confiabilidad de los sistemas pone en riesgo la integridad de datos sensibles, comprometiendo la confianza en la digitalización nacional y regional [6, 7]. Como advierte la literatura reciente, no se trata de errores sintácticos aislados, sino de una degradación estructural o *PromptDebt* que compromete directamente la resiliencia y la competitividad organizacional [4, 11].

De no establecerse un **Framework de Ingeniería de Prompts** que actúe como un filtro de calidad basado en la norma [2], la industria del software se enfrentará a una saturación de sistemas heredados (*legacy*) generados por IA que resultarán inmanejables a mediano plazo [1, 4]. Prospectivamente, el éxito de la ingeniería de sistemas en la próxima década dependerá de la capacidad de los profesionales para sistematizar la interacción con los LLMs, transformando la generación de código de un proceso aleatorio a una disciplina de ingeniería predictiva y controlada [13, 14]. Solo mediante la adopción de estrategias de gobernanza y métricas de calidad adaptadas a la era generativa se podrá asegurar que la

tecnología cumpla con los objetivos de transformación digital nacional, actuando como un motor de eficiencia y no como un generador de inestabilidad sistémica en infraestructuras críticas [6, 17].

La unidad de análisis está constituida por módulos de código fuente generados por modelos de lenguaje de gran escala (LLMs), específicamente implementaciones de microservicios desarrolladas en lenguajes de programación como Python y TypeScript. Cada unidad será evaluada mediante métricas de calidad interna para determinar su nivel de mantenibilidad y deuda técnica.

1.2 Formulación del Problema

1.2.1 Problema General

¿Qué efecto tiene la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa?

1.2.2 Problemas Específicos

PE1 ¿Qué efecto tiene la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe del código generado?

PE2 ¿Qué efecto tiene la implementación de un pipeline de análisis estático en la reducción de la deuda técnica del software generado?

PE3 ¿Qué efecto tiene la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad del código generado?

1.3 Objetivos de la Investigación

1.3.1 Objetivo General

Evaluar el efecto de la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa.

1.3.2 Objetivos Específicos

OE1 Evaluar el efecto de la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe.

OE2 Evaluar el efecto de la implementación de un pipeline de análisis estático en la reducción de la deuda técnica.

OE3 Evaluar el efecto de la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad.

1.4 Justificación de la Investigación

1.4.1 Justificación Teórica

La investigación se justifica teóricamente al proponer una convergencia necesaria entre la Ingeniería de Software clásica y el paradigma emergente de la Inteligencia Artificial Generativa (IAG). Ante el fenómeno de la çaja negra"que representan los LLMs, este estudio aporta un marco conceptual que vincula el determinismo de la norma ISO/IEC 25010 con la naturaleza estocástica del lenguaje natural. Al integrar la Teoría de la Complejidad de McCabe con la estructuración de instrucciones, se genera un nuevo conocimiento sobre cómo la semántica del *prompt* impacta directamente en el grafo de flujo de control del software resultante. Este trabajo expande la frontera del conocimiento al tratar a la IA no como un ente aislado, sino como un agente dentro de un sistema sociotécnico que requiere de gobernanza arquitectónica para mitigar la entropía del código.

1.4.2 Justificación Práctica

En la práctica, el estudio aborda la crisis de mantenibilidad generada por el uso empírico de la IAG en la industria. Al proporcionar un Framework validado, se ofrece una solución tangible al problema de la "PromptDebt", permitiendo que las organizaciones reduzcan el tiempo de refactorización manual y los costos operativos de soporte de nivel 3 [16]. La implementación de este marco transforma el uso de la IA de un asistente de autocompletado a un motor de generación de microservicios profesionalmente aptos para infraestructuras críticas, garantizando que el ahorro inicial en tiempo de codificación no se traduzca en una deuda técnica impagable a mediano plazo [9].

1.4.3 Justificación Metodológica

Metodológicamente, la investigación establece un precedente de auditoría técnica para el código sintético. Se justifica mediante el diseño de un *pipeline* de evaluación experimental que elimina la subjetividad en la apreciación de la calidad del código. Al combinar herramientas de análisis estático (como SonarQube) con protocolos de validación normativa, se propone un modelo de experimentación replicable y escalable. Este procedimiento permite transformar la Ïngeniería de Prompts"de una técnica de ensayo y error en una disciplina de ingeniería medible, auditable y controlada, proporcionando métricas precisas para la toma de decisiones en la gestión de proyectos tecnológicos.

1.5 Delimitación de la Investigación

1.5.1 Delimitación Espacial

El estudio se desarrollará en el Laboratorio de Ingeniería de Sistemas de la Universidad Continental, sede Cusco. La infraestructura tecnológica consistirá en estaciones de trabajo bajo el sistema operativo Arch Linux, configuradas para la gestión de contenedores y despliegue de microservicios. Se utilizarán modelos de lenguaje de última generación a través de APIs y entornos locales para garantizar la trazabilidad de los datos y minimizar la latencia de red, asegurando un entorno de pruebas controlado y aislado de interferencias externas.

1.5.2 Delimitación Temporal

La investigación es de corte transversal y se ejecutará durante el periodo académico 2026-I, abarcando desde marzo hasta julio de 2026. Este periodo permite una observación secuencial que inicia con la delimitación algorítmica [13], continúa con la fase de experimentación intensiva y concluye con el análisis estadístico de la deuda técnica acumulada frente a los umbrales de la norma ISO 25010.

1.5.3 Delimitación Social

El alcance social del estudio se circunscribe a los actores del ecosistema de desarrollo de software: ingenieros de sistemas, arquitectos de soluciones y especialistas en aseguramiento de la calidad (QA). Se enfoca particularmente en la comunidad técnica de la región Cusco, buscando profesionalizar el uso de la IAG en las Mypes tecnológicas y en la formación de futuros ingenieros, alineándose con las estrategias nacionales de transformación digital y ética en IA [6, 18].

1.6 Hipótesis y Variables

1.6.1 Hipótesis

Hipótesis General: La implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 reduce significativamente la deuda técnica y mejora la mantenibilidad del código generado por inteligencia artificial generativa en comparación con métodos empíricos.

Hipótesis Específicas:

HE1 El código generado mediante el uso de patrones arquitectónicos de prompts presenta menor complejidad ciclomática que el código generado mediante métodos empíricos.

HE2 El código generado con un pipeline de análisis estático presenta menor deuda técnica que el generado sin dicho pipeline.

HE3 El código generado bajo el protocolo de validación basado en ISO/IEC 25010

presenta un mayor índice de mantenibilidad que el código generado sin dicho protocolo.

1.6.2 Definición de Variables

Variable Independiente (X): Aplicación del Framework de Ingeniería de Prompts

- Nivel 1: Sin framework (método empírico)
- Nivel 2: Con framework estructurado

Variables Dependientes (Y):

VD1 **Mantenibilidad del Software:** Medida a través del índice de mantenibilidad y la complejidad ciclomática de McCabe utilizando herramientas de análisis estático (SonarQube/Radon).

VD2 **Deuda Técnica:** Medida a través del tiempo estimado de refactorización y la densidad de defectos detectados por el *pipeline* automatizado.

1.6.3 Operacionalización de Variables

Cuadro 1: Operacionalización de Variables

Variable	Dimensión	Indicador	Técnica	Instrumento
Framework de Prompts (VI)	Aplicación estructurada	Uso de patrones como Chain-of-Thought y Few-shot prompting	Experimental	Plantilla de prompts
Mantenibilidad (VD1)	Calidad interna	Índice de mantenibilidad	Análisis estático	SonarQube
	Complejidad	Complejidad ciclomática	Análisis estático	Radon
Deuda técnica (VD2)	Esfuerzo	Tiempo de remediación	Análisis automático	SonarQube
	Calidad	Densidad de defectos	Análisis automático	SonarQube

CAPÍTULO 2
MARCO TEÓRICO

2.1 Antecedentes de la investigación

2.1.1 Antecedentes Internacionales

Asfandyar, Akhtar y Khalil [9], en su estudio de 2024 titulado *Evaluating the Impact of AI Code Generated of Software Quality and Observed Study of Maintainability, Security and Performance*, analizaron el efecto de la automatización generativa en la integridad técnica de los sistemas. El objetivo principal fue evaluar la calidad del código producido por modelos de lenguaje en términos de mantenibilidad y seguridad. Mediante una metodología experimental de enfoque cuantitativo, se compararon métricas de software entre desarrollos humanos y sintéticos, utilizando herramientas de análisis estático sobre una muestra de diversos algoritmos. Los resultados revelaron que el código generado por inteligencia artificial presentó un incremento del 34 % en la complejidad ciclomática de McCabe y una mayor tasa de redundancia lógica. Los autores concluyeron que, sin una guía estructural, la IA degrada la arquitectura del software. Este estudio es fundamental para la investigación actual, ya que proporciona la base estadística para el planteamiento del problema y justifica el uso de la complejidad ciclomática como métrica de control en el primer objetivo específico.

Aljohani y Do [4], en su investigación de 2024 titulada *PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects*, examinaron la aparición de ineficiencias en proyectos que integran modelos de lenguaje. El estudio se propuso identificar y clasificar las consecuencias derivadas de instrucciones mal estructuradas. La metodología adoptada fue un análisis empírico descriptivo basado en la minería de repositorios de código abierto, evaluando interacciones entre desarrolladores y modelos generativos. Los hallazgos principales demostraron que la ausencia de estándares en la ingeniería de prompts genera una "deuda de prompt"(*PromptDebt*) que dificulta la evolución del sistema a largo plazo. Se concluyó que la gestión de esta deuda requiere marcos de gobernanza técnica específicos. El estudio resulta altamente relevante, pues introduce formalmente el concepto de deuda técnica aplicada a prompts, vinculándose directamente con la hipótesis de que un framework basado en ISO 25010 puede mitigar este impacto sistémico.

Azeem, Naveed, Sajid y Ali [15], desarrollaron en 2024 el estudio *AI vs. Human Programmers: Complexity and Performance in Code Generation*, con el propósito de contrastar la calidad del código generado por agentes de inteligencia artificial frente al desarrollado por humanos. La investigación empleó un diseño experimental y comparativo, aplicando métricas de complejidad de Halstead y McCabe sobre una población de algoritmos estándar. Los resultados indicaron que, aunque la IA posee

una velocidad de respuesta superior, el código resultante es significativamente más verboso y posee un acoplamiento superior al deseado. Los investigadores concluyeron que la tecnología generativa aún no cumple con los estándares industriales de calidad interna de forma autónoma. Este antecedente fundamenta la necesidad de un framework de ingeniería de prompts, alineándose con el objetivo general de evaluar el índice de mantenibilidad y la necesidad de establecer un protocolo de validación técnica para reducir la intervención manual post-generación.

Cordeiro, Noei y Zou [19], en su trabajo de 2024 titulado *An Empirical Study on the Code Refactoring Capability of Large Language Models*, investigaron la competencia de los modelos de lenguaje para optimizar código existente. El estudio buscó medir la efectividad de los modelos en la aplicación de patrones de refactorización. Se utilizó una metodología cuantitativa de diseño experimental, empleando técnicas de recolección de datos basadas en prompts de tipo *Few-shot learning*. Los hallazgos mostraron que el uso de instrucciones guiadas mejora la precisión de la refactorización en comparación con prompts simples. Se concluyó que la ingeniería de prompts es el factor determinante para la calidad del código arquitectónico. La relevancia de este estudio para la investigación actual reside en la validación metodológica de los patrones de prompts, lo cual sustenta el diseño del framework propuesto y el cumplimiento de los objetivos específicos relacionados con la reducción de la deuda técnica.

Cuatecontzi Cuahutle, Medina Barrera, Cortes Maldonado y Bueno Avendaño [12], realizaron en 2023 la revisión sistemática titulada *Identificación y Medición de Deuda Técnica Autoadmitida en Herramientas de Aprendizaje Profundo*. El objetivo fue categorizar los tipos de deuda técnica presentes en sistemas inteligentes. La metodología consistió en una revisión bibliográfica sistemática bajo un enfoque cualicuantitativo, analizando comentarios de código y registros de *commits*. Los resultados identificaron que la deuda técnica autoadmitida predomina en las fases de diseño y algoritmos, afectando la portabilidad y mantenibilidad. Los autores ofrecieron como aporte un esquema para identificar señales de degradación en el código sintético. Este antecedente se vincula con la presente investigación al proporcionar el marco teórico sobre la deuda técnica autoadmitida (SATD), variable fundamental para el segundo objetivo específico que busca reducir dicha deuda mediante un *pipeline* de evaluación automatizado.

Siddiq y Santos [8], en el estudio de 2023 *Exploring the Effectiveness of Large Language Models in Generating Unit Tests*, evaluaron la capacidad de la IA para asegurar la testabilidad del software. El objetivo fue medir la cobertura y calidad de las pruebas unitarias generadas por modelos generativos. Mediante una metodología experimental, se analizaron fragmentos de código, recolectando datos sobre la tasa de éxito de ejecución. Los resultados indicaron que la IA suele generar pruebas que fallan por falta de contexto arquitectónico. Se concluyó que la testabilidad —sub-

característica de la ISO 25010— es una de las áreas más críticas en la generación asistida. Este estudio es pertinente debido a que justifica la inclusión del protocolo de validación basado en la norma ISO 25010, permitiendo al investigador actual centrarse en la subcaracterística de capacidad de ser probado dentro del índice de mantenibilidad.

2.1.2 Antecedentes Nacionales

Romero Aguilar, Vargas Alarcón, Balboa Zegarra, Alvarado Sanchez y Plasencia Sifuentes [16], en su investigación de 2024 titulada *Business Consulting: Aplicaciones de marcos de referencia para el desarrollo de software en MS4M S.A.C.*, analizaron la problemática de la deuda técnica en el sector tecnológico peruano. El objetivo fue proponer lineamientos de gobernanza para reducir costos operativos de mantenimiento. La metodología fue un estudio de caso descriptivo con enfoque cualitativo, recolectando datos mediante diagnósticos técnicos en una empresa de software minero. Los resultados mostraron que la falta de arquitectura estandarizada incrementa los tiempos de soporte significativamente. Se concluyó que la implementación de marcos de referencia arquitectónicos es vital para la sostenibilidad. Este antecedente nacional es relevante ya que contextualiza la variable de deuda técnica en el mercado peruano, respaldando la justificación práctica y económica de la investigación actual sobre microservicios.

Pardo Mesias [20], en su estudio de 2024 denominado *Mantenibilidad de productos de software según el modelo SQUARE ISO/IEC 25000*, investigó el nivel de calidad interna en desarrollos realizados por egresados de ingeniería en el Perú. El estudio se propuso medir el grado de mantenibilidad aplicando la norma ISO/IEC 25000. Utilizando una metodología aplicada y descriptiva, evaluó una muestra de aplicaciones mediante herramientas de análisis estático. Los hallazgos revelaron deficiencias críticas en modularidad y capacidad de ser probado. El aporte principal fue una estrategia de medición estandarizada para el entorno académico nacional. Este estudio se relaciona directamente con la investigación presente al validar el uso del modelo ISO 25010 como el estándar adecuado para medir la variable dependiente, proporcionando un punto de comparación regional para los resultados de mantenibilidad que se obtendrán en la experimentación.

Díaz Sifuentes, Paz Fernández y Torres Villanueva [14], en el artículo de 2024 titulado *Optimización de Modelos de Lenguaje Grande (LLMs) a través del Prompt Engineering*, exploraron técnicas para mejorar la precisión de la inteligencia artificial. El estudio tuvo como objetivo evaluar el impacto del *Few-shot learning* en la reducción de errores lógicos. La metodología fue experimental y comparativa, analizando el rendimiento de modelos generativos bajo diferentes configuraciones de instrucciones. Los resultados demostraron que un diseño estratégico de prompts reduce las alucinaciones y mejora la relevancia técnica de las respuestas. Se con-

cluyó que el *prompt engineering* actúa como una capa de optimización externa fundamental. Para la investigación actual, este antecedente es esencial pues sustenta metodológicamente la variable independiente, demostrando que la estructura del prompt es el medio idóneo para controlar la calidad del software generado.

Medianero Acosta [13], en el trabajo de 2024 titulado *Algoritmos imperativos como insumo para generar código de programación aplicando técnicas de Ingeniería Rápida en Inteligencia Artificial*, investigó el uso de lógica estructurada como base para prompts. El objetivo fue optimizar el proceso de codificación mediante la autogeneración asistida por modelos LLM. La metodología empleada fue de carácter técnico-aplicado, integrando algoritmos imperativos como fuente sintáctica de entrada para la generación de código profesional. Los resultados mostraron una mejora en la eficiencia del desarrollo y una reducción en los recursos de producción. Se concluyó que el uso de algoritmos estructurados redefine la construcción de software profesional. Este aporte es vital para la investigación presente, ya que valida el uso de algoritmos como insumo dentro del framework propuesto, alineándose con el objetivo de reducir la complejidad ciclomática desde la fase de instrucción.

Gutiérrez Chávez [11], en la investigación de 2024 denominada *Análisis de la aplicación de prácticas ágiles y la gestión de la deuda técnica en el desarrollo de software en MediaLab*, evaluó cómo los plazos ajustados afectan la sostenibilidad del código. El estudio se propuso identificar las causas de la baja mantenibilidad en entornos de transformación digital. Mediante una metodología cuantitativa y descriptiva, se combinaron encuestas y análisis con SonarQube sobre proyectos activos. Los resultados indicaron que la ausencia de estrategias de gestión de deuda técnica incrementa significativamente los costos operativos. Se concluyó que se requiere un enfoque preventivo en la planificación para mejorar la calidad del software. Este estudio respalda la investigación actual al justificar el uso de un pipeline de análisis estático para la detección temprana de deuda, vinculándose con el segundo objetivo específico de la tesis.

2.2 Bases teóricas

2.2.1 Marco Conceptual

2.2.1.1 Ingeniería de Prompts (Prompt Engineering)

Disciplina técnica centrada en el diseño y optimización de instrucciones textuales para guiar modelos de lenguaje hacia resultados específicos. Implica el uso de patrones lógicos que minimizan la ambigüedad sintáctica, permitiendo que la inteligencia artificial genere soluciones alineadas con requerimientos arquitectónicos complejos de software.

2.2.1.2 Mantenibilidad del Software

Atributo de calidad que define la facilidad con la que un sistema puede ser modificado, corregido o mejorado. Según la normativa internacional, comprende subcaracterísticas esenciales como la modularidad y la capacidad de ser probado, asegurando que el código sea sostenible durante su ciclo de vida.

2.2.1.3 Deuda Técnica (Technical Debt)

Costo implícito derivado de priorizar la rapidez de entrega sobre la calidad del diseño estructural. Se manifiesta como la acumulación de soluciones subóptimas que requieren refactorización futura, incrementando la complejidad sistémica y los costos operativos de mantenimiento en entornos de desarrollo profesional de software.

2.2.1.4 Complejidad Ciclomática de McCabe

Métrica cuantitativa utilizada para determinar la complejidad lógica de un programa mediante el recuento de rutas linealmente independientes en su flujo de control. Actúa como un indicador crítico de la mantenibilidad, prediciendo la dificultad de evaluar y modificar fragmentos de código específicos en sistemas informáticos.

2.2.1.5 PromptDebt

Fenómeno emergente en la ingeniería generativa donde la carencia de estándares en las instrucciones deriva en código con alta redundancia y baja cohesión. Representa una degradación estructural que compromete la evolución del software, exigiendo marcos de gobernanza técnica para mitigar su impacto en la infraestructura crítica.

2.2.1.6 Análisis Estático de Código

Técnica de inspección automatizada que evalúa el código fuente sin ejecutarlo para identificar vulnerabilidades y desviaciones de estándares de calidad. Permite la detección temprana de defectos estructurales y la medición precisa de métricas de mantenibilidad mediante herramientas especializadas como SonarQube o Radon.

2.2.1.7 Deuda Técnica Autoadmitida (SATD)

Categoría de deuda técnica donde las deficiencias arquitectónicas son reconocidas y documentadas explícitamente por los desarrolladores mediante comentarios o metadatos. En sistemas generados por inteligencia artificial,

indica áreas críticas que requieren intervención humana obligatoria para asegurar la portabilidad y eficiencia del producto final.

2.2.2 Bases Teóricas

2.2.2.1 Ingeniería de Software y Calidad del Producto

- **Modelos de calidad de software y evolución:** La ingeniería de software ha evolucionado desde un enfoque centrado en la ejecución hacia la gestión de atributos internos que garantizan la sostenibilidad del producto. Modelos clásicos como los de McCall y Boehm permitieron por primera vez descomponer la çalidad.[en] factores medibles [20]. Esta evolución técnica es la base que permite hoy evaluar si el código generado por inteligencia artificial cumple con los requisitos mínimos de robustez arquitectónica necesarios en entornos de producción industrial.
- **La norma ISO/IEC 25010 y la mantenibilidad:** El estándar ISO/IEC 25010 constituye el eje normativo de este estudio, definiendo la mantenibilidad como la capacidad de un sistema para ser modificado efectiva y eficientemente [2]. Se enfoca en subcaracterísticas clave: modularidad, reusabilidad, analizabilidad, capacidad de ser modificado y capacidad de ser probado. Estas dimensiones permiten cuantificar el impacto de la ingeniería de prompts, determinando si el código resultante facilita o dificulta la evolución del sistema frente a cambios de requisitos.
- **Métricas de calidad interna y complejidad:** Para validar la mantenibilidad, se emplean métricas cuantitativas como la Complejidad Ciclomática de McCabe y el Índice de Mantenibilidad (MI). Estos indicadores evalúan la densidad lógica y el esfuerzo cognitivo requerido para comprender el código [9]. En el desarrollo de microservicios, mantener estos valores dentro de umbrales óptimos es crucial para evitar el colapso del sistema por acumulación de errores arquitectónicos invisibles en la superficie funcional.

2.2.2.2 Inteligencia Artificial Generativa en el Desarrollo de Software

- **Modelos de Lenguaje de Gran Escala (LLMs):** Los modelos fundamentados en arquitecturas de transformadores han revolucionado la programación mediante la predicción probabilística de tokens de código. Estos sistemas, entrenados en vastos repositorios como GitHub, poseen una capacidad emergente para resolver problemas algorítmicos complejos [1]. Sin embargo, su naturaleza estocástica implica que la calidad de la salida es altamente dependiente del contexto proporcionado, lo que genera una variabilidad técnica que debe ser controlada mediante marcos de referencia.
- **Capacidades y limitaciones en la generación de lógica:** Aunque los

LLMs demuestran una alta velocidad en la creación de funciones, suelen presentar limitaciones en la visión arquitectónica global y la cohesión de módulos extensos [15]. El riesgo de generar código "verboso.º con redundancia lógica es una constante que afecta la deuda técnica. Esta limitación justifica la necesidad de un framework que no solo genere código, sino que condicione la generación a estándares de calidad prefijados.

2.2.2.3 Paradigma de la Ingeniería de Prompts (Prompt Engineering)

- **Técnicas de estructuración de instrucciones:** La ingeniería de prompts se define como la disciplina de optimizar las entradas de un modelo para obtener salidas precisas. Técnicas como el *Few-Shot Prompting* permiten al modelo aprender de ejemplos previos insertados en la instrucción, mejorando la adherencia a estilos de codificación específicos [14]. Estas técnicas son el insumo principal de la variable independiente en la presente investigación, actuando como el mecanismo de control de calidad.
- **Patrones arquitectónicos de prompts:** El uso de patrones como *Chain-of-Thought* (CoT) obliga al modelo a razonar.ª través de pasos intermedios antes de emitir el código final. Este desglose lógico reduce las alucinaciones técnicas y asegura que la lógica de negocio se traduzca correctamente a código ejecutable [13]. La integración de estos patrones en un framework estructurado permite una generación de microservicios más predecible y alineada con las mejores prácticas de la industria.

2.2.2.4 Deuda Técnica en Entornos Generativos

- **Teoría de la Deuda Técnica y evolución:** La deuda técnica es la metáfora financiera aplicada al software que representa el costo futuro de las decisiones de diseño apresuradas [11]. En la era de la IA, esta deuda se acelera debido a la capacidad de los modelos para producir grandes volúmenes de código subóptimo en segundos. La gestión proactiva de esta deuda es el objetivo central del segundo componente experimental de este estudio.
- **PromptDebt y opacidad estructural:** La "Deuda de Prompt"surge cuando las instrucciones utilizadas para generar el sistema no son sostenibles ni escalables [4]. Esto genera una opacidad en el código fuente que dificulta su mantenimiento futuro, ya que el desarrollador humano debe lidiar con decisiones de diseño tomadas por un modelo sin documentación clara. El estudio busca mitigar este riesgo mediante la estandarización de la instrucción bajo la norma ISO 25010.

2.2.2.5 Marcos de Trabajo y Evaluación Automatizada

- **Frameworks de gobernanza técnica:** Un framework de ingeniería de prompts actúa como una capa intermedia de gobernanza que garantiza que cada línea de código generada sea evaluada bajo criterios de calidad antes de su integración [17]. Esta estructura permite automatizar la aplicación de reglas arquitectónicas, reduciendo la dependencia del juicio humano individual y asegurando la uniformidad en sistemas críticos de microservicios.
- **Pipelines de análisis estático:** La integración de herramientas de análisis estático en el flujo de desarrollo permite medir en tiempo real el Índice de Mantenibilidad y la Deuda Técnica. El uso de herramientas especializadas permite capturar señales premonitorias de degradación técnica [12]. Esta automatización de la vigilancia de calidad es el componente final del framework propuesto, cerrando el ciclo entre la instrucción (prompt) y la validación normativa.

CAPÍTULO 3
METODOLOGÍA

3.1 Método, tipo y alcance de la investigación

3.1.1 Método de investigación

El método de investigación empleado es el hipotético-deductivo. Este método permite partir de una premisa teórica basada en la ingeniería de software y la norma ISO/IEC 25010, para luego someterla a comprobación mediante la observación empírica de métricas de código. A través de la deducción, se establecen las hipótesis que serán validadas o refutadas mediante el análisis estático de los resultados obtenidos tras la aplicación del Framework de Ingeniería de Prompts propuesto.

3.1.2 Tipo de investigación

La investigación es de tipo aplicada y tecnológica. Es aplicada porque busca resolver un problema concreto de la industria del software: la acumulación de deuda técnica en sistemas generados por IA. Es tecnológica porque el aporte principal consiste en la creación de un Framework (herramienta metodológica) y un pipeline de evaluación automatizado que optimiza el proceso de producción de software, transformando el conocimiento teórico en una solución técnica ejecutable.

3.1.3 Alcance de la investigación

De acuerdo con la Guía Metodológica, el alcance de esta investigación es explicativo. El estudio no se limita a describir cómo la IA genera código o a correlacionar variables, sino que pretende establecer una relación de causalidad. Se busca determinar cómo y por qué la estructuración sistémica de los prompts influye directamente en la reducción de la complejidad ciclomática y en la mejora de los índices de mantenibilidad del producto final.

3.1.4 Diseño de la investigación

El diseño de la investigación es experimental, específicamente bajo un esquema cuasi-experimental con pre-prueba y post-prueba. Se utilizará un grupo de control donde la generación de código se realizará mediante métodos empíricos (prompts abiertos) y un grupo experimental donde se aplicará el Framework de Ingeniería de Prompts diseñado. Las mediciones de pre-prueba y post-prueba se realizarán utilizando herramientas de análisis estático (SonarQube/Radon) para comparar la evolución de la calidad estructural del software en ambos escenarios.

3.1.5 Enfoque de la investigación

La presente investigación adopta un enfoque cuantitativo, debido a que se fundamenta en la medición objetiva de variables mediante métricas de calidad de software, tales como el índice de mantenibilidad, la complejidad ciclomática y la deuda técnica. Los datos obtenidos serán analizados estadísticamente con el fin de contrastar las hipótesis planteadas y determinar relaciones de causalidad entre la variable independiente y las variables dependientes.

3.2 Materiales y métodos

3.2.1 Materiales

3.2.2 Recursos de Infraestructura y Hardware

3.2.2.1 Estación de Trabajo de Alto Rendimiento

Se refiere a la unidad de procesamiento centralizada destinada a la ejecución de tareas de cómputo intensivo requeridas para la inferencia de modelos de lenguaje y el análisis estático de código.

Componente	Especificación Técnica
Arquitectura	x86_64
Procesador	Multinúcleo (Multi-core)
Memoria RAM	Mínimo 16 GB

Cuadro 2: Especificaciones técnicas de la estación de trabajo.

Función dentro del sistema: Actúa como el host principal para el despliegue de contenedores y la gestión de flujos de datos entre los modelos de IA y las herramientas de métricas.

Uso en la solución: Provee la potencia de cálculo necesaria para procesar las 50 peticiones de generación de código por grupo y ejecutar simultáneamente los servicios de SonarQube.

3.2.2.2 Entorno de Ejecución y Nodos Computacionales

Definido como el ecosistema de software de bajo nivel y red local configurado para el control estricto de variables de procesamiento.

- **Sistema Operativo:** Distribución Linux (específicamente Arch Linux para el clúster de pruebas).
- **Infraestructura:** Nodos computacionales locales configurados en el Laboratorio de Ingeniería de Sistemas de la Universidad Continental, sede Cusco.

Función dentro del sistema: Garantiza un entorno de ejecución aislado que minimiza la latencia de red y asegura la reproducibilidad de los experimentos.

Relación con la arquitectura: Constituye la capa de infraestructura (On-premise) sobre la cual se despliega el clúster de pruebas y los servicios de análisis.

3.2.3 Recursos Lógicos y Modelos de Inteligencia Artificial

3.2.3.1 Modelos de Lenguaje de Gran Escala (LLMs) de Acceso Abierto

Constituyen los motores de inferencia probabilística basados en arquitecturas de transformadores, especializados en la generación de sintaxis y lógica de programación mediante el procesamiento de lenguaje natural.

Modelo	Versión/Parámetros	Aplicación Principal
Llama 3	8B / 70B	Generación de microservicios y lógica compleja.
Mistral	v0.1 / v0.3	Validación de reproducibilidad y consistencia.

Cuadro 3: Modelos de lenguaje seleccionados para la investigación.

Función dentro del sistema: Actúan como el agente generador de código fuente que será sometido a evaluación bajo las métricas de mantenibilidad y deuda técnica.

Uso en la solución: Reciben las instrucciones (prompts) estructuradas bajo los lineamientos del framework basado en la norma ISO/IEC 25010 para producir módulos de microservicios.

Lenguaje y entorno de desarrollo: El experimento se desarrollará utilizando el lenguaje de programación Python, debido a su amplia adopción en entornos de desarrollo asistido por inteligencia artificial y su compatibilidad con herramientas de análisis estático como Radon y SonarQube. Asimismo, se empleará un enfoque basado en microservicios utilizando frameworks ligeros como FastAPI para la generación de módulos evaluables.

3.2.3.2 Entorno de Orquestación y Contenerización

Capa de virtualización a nivel de sistema operativo utilizada para empaquetar y ejecutar los servicios de análisis y modelos de forma aislada y replicable.

- **Motor de contenedores:** Docker para el aislamiento de servicios.

- **Orquestación:** Gestión de dependencias y despliegue de SonarQube y Radon.

Relación con la arquitectura: Permite la integración del pipeline de análisis estático con los nodos computacionales, asegurando que el entorno de experimentación sea independiente del hardware subyacente.

3.2.4 Herramientas de Análisis Estático y Desarrollo

3.2.4.1 Framework de Evaluación de Calidad de Código

Conjunto de herramientas de software destinadas a la inspección automática del código fuente sin necesidad de ejecución, permitiendo identificar vulnerabilidades estructurales.

- **SonarQube (Community Edition):** Plataforma central para la medición de la deuda técnica y densidad de defectos. Para el experimento, se configurará un *Quality Gate* estricto, definiendo como métrica de fallo cualquier módulo generado que supere un ratio de deuda técnica del 5 % o que presente vulnerabilidades críticas en el atributo de mantenibilidad.
- **Librería Radon:** Herramienta de línea de comandos en Python para el cálculo de la Complejidad Ciclomática de McCabe.

3.2.4.2 Ecosistema de Lenguajes de Programación Objetivo

Lenguajes de alto nivel seleccionados para la generación de código, representativos del desarrollo actual de microservicios:

- **Python:** Utilizado para la generación de lógica de procesamiento de datos.
- **TypeScript:** Utilizado para la validación de arquitectura de servicios escalables y tipado fuerte.

3.2.5 Planificación del Proyecto

3.2.5.1 Presupuesto

El presupuesto del proyecto ha sido estructurado considerando los recursos necesarios para la ejecución del proceso experimental, incluyendo recursos humanos, infraestructura, servicios y costos operativos. Esta distribución permite garantizar la viabilidad técnica y económica del estudio.

Categoría	Ítem	Cant.	Unidad	P. Unit (S/.)	Total (S/.)
Recursos Humanos	Investigador principal	6	mes	0	0
	Asistente de investigación	0	mes	0	0
Equipamiento	Estación de trabajo	1	unidad	0	0
	Periféricos	1	lote	200	200
Software	Herramientas open source	1	uso	0	0
	Consumo API LLM	1	uso	0	0
Servicios	Internet	6	mes	100	600
	Electricidad	6	mes	80	480
Otros	Impresión	1	lote	150	150
				TOTAL	**1,430**

Cuadro 4: Estructura del presupuesto ajustado del proyecto.

El experimento utilizará modelos de lenguaje de acceso gratuito o con planes académicos, lo cual elimina costos asociados al consumo de API, manteniendo la viabilidad económica del estudio. El proyecto será ejecutado por el investigador como parte de su formación académica, por lo que no se consideran costos salariales.

Criterio de estimación: El presupuesto prioriza el componente de recursos humanos, dado que la investigación se fundamenta en actividades de análisis, diseño experimental y validación. Los costos de software se mantienen reducidos debido al uso de herramientas open source, mientras que se considera un margen de contingencia para cubrir variaciones operativas.

3.2.5.2 Cronograma de Actividades

El cronograma del proyecto ha sido estructurado en un horizonte temporal de 24 semanas, comprendido entre julio y diciembre de 2026. Se ha definido una secuencia lógica que integra actividades de investigación, desarrollo experimental y validación, permitiendo la superposición controlada de tareas para optimizar el tiempo de ejecución.

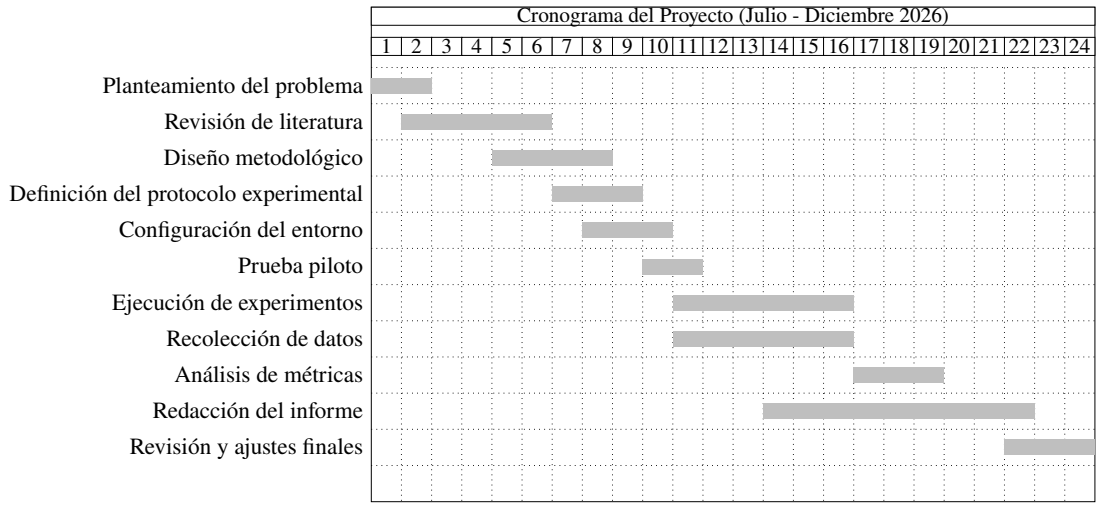

Figura 1: Cronograma del proyecto

Criterio de planificación: Se ha optado por un enfoque semi-paralelo en la ejecución de actividades críticas, particularmente en la fase experimental,

donde la generación de código y la recolección de datos se desarrollan de manera concurrente. Esta estrategia reduce el tiempo total del proyecto sin comprometer la validez de los resultados.

3.2.6 Métodos

3.2.7 Ingeniería de Prompts y Diseño del Framework

3.2.7.1 Conceptualización del Framework basado en ISO/IEC 25010

Proceso de traducción de los atributos de calidad definidos en la norma internacional hacia una estructura lógica de instrucciones para modelos de lenguaje. Se centra en la descomposición de la Mantenibilidad en sub-características como modularidad y analizabilidad.

- **Justificación:** Garantiza que el código generado cumpla con estándares industriales para su evolución futura.
- **Artefactos:** Matriz de trazabilidad Norma-Prompt y Plantilla Maestra del Framework.

3.2.7.2 Aplicación de Patrones Arquitectónicos de Prompts

Implementación de técnicas avanzadas de estructuración para guiar el razonamiento del modelo de IA hacia soluciones óptimas.

- **Chain-of-Thought (CoT):** Descomposición de problemas lógicos en pasos secuenciales.
- **Few-Shot Prompting:** Inclusión de ejemplos de código de alta calidad como referencia arquitectónica.

3.2.8 Desarrollo y Generación de Código de Microservicios

3.2.8.1 Configuración del Entorno de Generación

Preparación de la infraestructura lógica para la ejecución de modelos en un entorno controlado. Incluye el despliegue de contenedores Docker y la calibración de hiperparámetros como la temperatura (0.2) para asegurar determinismo.

3.2.8.2 Ejecución de Pruebas de Generación

Ejecución del protocolo cuasi-experimental dividido en dos fases:

- **Grupo Control (Pre-prueba):** Generación de 50 módulos utilizando prompts empíricos abiertos.

- **Grupo Experimental (Post-prueba):** Generación de módulos aplicando el Framework estructurado.

3.2.9 Pipeline de Análisis Estático y Validación de Calidad

3.2.9.1 Proceso de Evaluación de Métricas de Código

Implementación de un flujo automatizado para la extracción de metadatos técnicos. Se utiliza Radon para la Complejidad Ciclomática y SonarQube para el índice de mantenibilidad y deuda técnica.

3.2.9.2 Validación y Control de Calidad

Procedimiento de verificación del cumplimiento normativo y análisis de significancia estadística para la contrastación de las hipótesis planteadas.

- **Pruebas de Normalidad:** Previo al análisis comparativo, se aplicará la prueba de Shapiro-Wilk (considerando el tamaño de muestra $n = 50$ por grupo) para determinar si los datos métricos presentan una distribución normal.
- **Técnicas de Contraste:** Con base en los resultados de normalidad, se utilizará la prueba paramétrica t-Student (si existe normalidad) o la prueba no paramétrica U de Mann-Whitney (si no existe normalidad) para comparar las medias de los grupos control y experimental.
- **Criterio de Ingeniería:** Asegurar la sostenibilidad tecnológica mediante la reducción del tiempo estimado de remediación.

3.3 Sustento Teórico de la Metodología

La integración de la norma ISO/IEC 25010 dentro de la ingeniería de prompts se fundamenta en la necesidad de establecer métricas de calidad interna en sistemas generados por IA. Según Hou et al. (2024), la ausencia de rigor arquitectónico en el uso de LLMs deriva en una deuda técnica acumulada que compromete la mantenibilidad a largo plazo.

A nivel regional, la adopción de estas tecnologías en el sur del Perú requiere marcos de cumplimiento que aseguren la resiliencia organizacional frente a la degradación del software. El uso de herramientas como SonarQube y Radon permite operacionalizar estos estándares internacionales en datos cuantitativos y auditables. El enfoque cuantitativo y el diseño experimental adoptado se sustentan en lo planteado por Hernández-Sampieri et al.[21], quienes establecen que los estudios explicativos requieren la manipulación de variables independientes para observar su efecto en variables dependientes bajo condiciones controladas. Asimismo, el diseño cuasi-experimental permite trabajar con grupos no aleatorizados manteniendo validez interna en contextos aplicados.

CAPÍTULO 4

RESULTADOS Y DISCUSIÓN

4.1 Presentación de resultado

4.2 Discusión de resultados

CAPÍTULO 5

CONCLUSIONES Y RECOMENDACIONES

ANEXOS

Título: Evaluación del efecto de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa

Problemas	Objetivos	Hipótesis	Variables	Metodología
Problema General: ¿Qué efecto tiene la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa? **Problemas Específicos:** ▪ **PE1:** ¿Qué efecto tiene la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe del código generado? ▪ **PE2:** ¿Qué efecto tiene la implementación de un pipeline de análisis estático en la reducción de la deuda técnica del software generado? ▪ **PE3:** ¿Qué efecto tiene la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad del código generado?	**Objetivo General:** Evaluar el efecto de la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa. **Objetivos Específicos:** ▪ **OE1:** Evaluar el efecto de la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe. ▪ **OE2:** Evaluar el efecto de la implementación de un pipeline de análisis estático en la reducción de la deuda técnica. ▪ **OE3:** Evaluar el efecto de la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad.	**Hipótesis General:** La implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 reduce significativamente la deuda técnica y mejora la mantenibilidad del código generado por inteligencia artificial generativa en comparación con métodos empíricos. **Hipótesis Específicas:** ▪ **HIP1:** El código generado mediante el uso de patrones arquitectónicos de prompts presenta menor complejidad ciclomática que el código generado mediante métodos empíricos. ▪ **HIP2:** El código generado con un pipeline de análisis estático presenta menor deuda técnica que el generado sin dicho pipeline. ▪ **HIP3:** El código generado bajo el protocolo de validación basado en ISO/IEC 25010 presenta un mayor índice de mantenibilidad que el código generado sin dicho protocolo.	**Variable Independiente:** ▪ Aplicación del Framework de Ingeniería de Prompts • Nivel 1: Sin framework (método empírico) • Nivel 2: Con framework estructurado **Variables Dependientes:** ▪ Mantenibilidad del Software: • Índice de mantenibilidad • Complejidad ciclomática ▪ Deuda Técnica: • Tiempo de remediación • Densidad de defectos	**Método:** Hipotético-deductivo. **Tipo:** Aplicada / Tecnológica. **Nivel:** Explicativo con enfoque experimental. **Diseño:** Cuasi-experimental (grupo control vs grupo experimental). **Población/Muestra:** Módulos de lógica para microservicios generados por modelos de lenguaje (n = 50 muestras por grupo). **Técnicas/Instrumentos:** Experimentación y análisis estático de código mediante SonarQube y Radon.

REFERENCIAS BIBLIOGRÁFICAS

1. HOU, Xinyi; ZHAO, Yanjie; LIU, Yue; YANG, Zhou; WANG, Kailong; LI, Li; LUO, Xiapu; LO, David; GRUNDY, John y WANG, Haoyu. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* [online]. 2024, volumen 33, número 8, págs. 1-79 [visitado 2026-04-29]. ISSN 1049-331X, ISSN 1557-7392. Disponible desde DOI: 10.1145/3695988.

2. ISO/IEC. *ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Models*. International Organization for Standardization, 2011.

3. LIU, Yue; LE-CONG, Thanh; WIDYASARI, Ratnadira; TANTITHAMTHAVORN, Chakkrit; LI, Li; LE, Xuan-Bach D. y LO, David. Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues. *ACM Transactions on Software Engineering and Methodology* [online]. 2024, volumen 33, número 5, págs. 1-26 [visitado 2026-04-29]. ISSN 1049-331X, ISSN 1557-7392. Disponible desde DOI: 10.1145/3643674.

4. ALJOHANI, Ahmed y DO, Hyunsook. PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects. En: *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering* [online]. Istanbul Turkiye: ACM, 2025, págs. 371-382 [visitado 2026-04-29]. ISBN 979-8-4007-1385-9. Disponible desde DOI: 10.1145/3756681.3756976.

5. INSTITUTO NACIONAL DE ESTADÍSTICA E INFORMÁTICA. *Informe Técnico: Estadísticas de las Tecnologías de Información y Comunicación en las Empresas*. Lima, Perú, 2024. Informe técnico. Instituto Nacional de Estadística e Informática.

6. SGTD. *Propuesta de la Estrategia Nacional de Inteligencia Artificial 2026-2030*. 2024. Informe técnico. Presidencia del Consejo de Ministros. Disponible también desde: https://www.gob.pe/institucion/pcm/informes-publicaciones. Secretaría de Gobierno y Transformación Digital.

7. GOBIERNO REGIONAL DE CUSCO. *Plan de Gobierno Digital 2025-2027*. Cusco, Perú, 2024. Informe técnico. Gerencia Regional de Planeamiento, Presupuesto y Modernización.

8. SIDDIQ, Mohammed Latif; SANTOS, Joanna C. S.; TANVIR, Ridwanul Hasan; ULFAT, Noshin; RIFAT, Fahmid Al y LOPES, Vinicius Carvalho. Using Large Language Models to Generate JUnit Tests: An Empirical Study. En: *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* [online]. 2024, págs. 313-322 [visitado 2026-04-29]. Disponible desde DOI: 10.1145/3661167.3661216. arXiv:2305.00418 [cs].

9. ASFANDYAR, Muhammad Waseem Akhtar. EVALUATING THE IMPACT OF AI CODE GENERATED OF SOFTWARE QUALITY AND OBSERVED STUDY OF MAINTAINABILITY, SECURITY AND PERFORMANCE. *Spectrum of Engineering Sciences* [online]. 2025, volumen 3, número 12, págs. 1379-1389 [visitado 2026-04-29]. ISSN 3007-312X. Disponible desde DOI: 10.5281/ZENODO.18172279.

10. FORMGREN, Björn. *Capabilities of Large Language Models in Generating Complex Code: A Software Architecture Perspective*. Estocolmo, Suecia, 2024. Tesis de maestría. KTH Royal Institute of Technology.

11. GUTIERREZ CHAVEZ, Hector Gonzalo. *Análisis de la aplicación de prácticas ágiles y la gestión de la deuda técnica en el desarrollo de software en MediaLab*. Tacna, Perú, 2025. Trabajo de Investigación de para optar el Grado a Nombre de la Nación de: Maestro en Diseño y Gestión de Proyectos Tecnológicos. Escuela de Posgrado Newman.

12. CUATECONTZI CUAHUTLE, Elizabeth; MEDINA BARRERA, María Guadalupe; CORTÉS MALDONADO, Raúl y BUENO AVENDAÑO, Carlos. Identificación y Medición de Deuda Técnica Autoadmitida en Herramientas de Aprendizaje Profundo: Una Revisión Sistemática. *Innovation and Software* [online]. 2025, volumen 6, número 2, págs. 171-191 [visitado 2026-04-29]. ISSN 2708-0935, ISSN 2708-0927. Disponible desde DOI: `10.48168/innosoft.s24.a287`.

13. MEDIANERO ACOSTA, Jorge. *Algoritmos imperativos como insumo para generar código aplicando técnicas de Prompt Engineering*. Lima, Perú, 2024. Trabajo de Suficiencia Profesional para optar al Título de Ingeniero de Sistemas y Cómputo. Universidad Inca Garcilaso de la Vega.

14. PAZ FERNÁNDEZ, Crishtian Brenon; DIAZ SIFUENTES, Sergio Helí y TORRES VILLANUEVA, Marcelino. Optimización de Modelos de Lenguaje Grande (LLMs) a través del Prompt Engineering. *Innovation and Software* [online]. 2025, volumen 6, número 2, págs. 6-11 [visitado 2026-04-29]. ISSN 2708-0935, ISSN 2708-0927. Disponible desde DOI: `10.48168/innosoft.s24.a212`.

15. AZEEM, Samina; NAVEED, Muhammad Shumail; SAJID, Muhammad y ALI, Imran. AI vs. Human Programmers: Complexity and Performance in Code Generation. *VAWKUM Transactions on Computer Sciences* [online]. 2025, volumen 13, número 1, págs. 201-216 [visitado 2026-04-29]. ISSN 2308-8168, ISSN 2411-6335. Disponible desde DOI: `10.21015/vtcs.v13i1.2043`.

16. ROMERO AGUILAR, Carmela; VARGAS ALARCÓN, Andric Alejandro; BALBOA ZEGARRA, Richar Sandro; ALVARADO SANCHEZ, Victor y PLASENCIA SIFUENTES, Victor Hugo. *Business Consulting: Aplicaciones de marcos de referencia para el desarrollo de software en MS4M S.A.C.* 2024. Tesis de Maestría. Pontificia Universidad Católica del Perú.

17. ORTEGA OVALLE, Maria Teodolinda. Arquitecturas basadas en microservicios: impacto en la mantenibilidad y la evolución de sistemas críticos: Microservices-based architectures: impact on maintainability and the evolution of critical systems. *LATAM Revista Latinoamericana de Ciencias Sociales y Humanidades* [online]. 2026, volumen 7, número 2 [visitado 2026-04-29]. ISSN 2789-3855. Disponible desde DOI: `10.56712/latam.v7i2.5535`.

18. GOBIERNO DEL PERÚ. *Estrategia Nacional de Inteligencia Artificial 2021-2026: Informe de Progreso y Actualización*. Lima, Perú, 2024. Informe técnico. Presidencia del Consejo de Ministros.

19. CORDEIRO, Jonathan; NOEI, Shayan y ZOU, Ying. An Empirical Study on the Code Refactoring Capability of Large Language Models. *ACM Transactions on Software Engineering and Methodology* [online]. 2026, pág. 3801158 [visitado 2026-04-29]. ISSN 1049-331X, ISSN 1557-7392. Disponible desde DOI: 10.1145/3801158.

20. PARDO MESIAS, Samuel Ricardo. *Mantenibilidad de productos de software según el modelo SQUARE ISO/IEC 25000*. Tingo María, Perú, 2018. Tesis de Pregrado. Universidad Nacional Agraria de la Selva.

21. HERNÁNDEZ SAMPIERI, Roberto; FERNÁNDEZ COLLADO, Carlos y BAPTISTA LUCIO, María del Pilar. *Metodología de la Investigación*. 6.ª edición. México D.F.: McGraw-Hill / Interamericana Editores, S.A. de C.V., 2014. ISBN 978-1-4562-2396-0.